Reviewer Pack — Minimal Eta-Invariant and Communication Complexity Rank Corr…

Entry b6ce94d7be22 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 03:24:47 UTC

Minimal Eta-Invariant and Communication Complexity Rank Correlation

Entry ID: `b6ce94d7be22` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-04 03:24:47 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Local Indeterminacy Theory) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

> For any given Boolean function f with communication complexity rank r(f), the minimal eta-invariant η(f) of the associated algebraic variety V_f is linearly correlated with r(f), such that η(f) = Θ(r(f)).

Rationale (proposer's reasoning):

> Local indeterminacy theory in algebraic geometry has not been widely applied to communication complexity, which could potentially reveal new structural insights. If the eta-invariant can be shown to correlate with the rank of the associated matrix, it might provide a bridge between geometric and combinatorial properties.

Taxonomy category: `communication_complexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `bca632be8480e51f`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the Pearson correlation coefficient between minimal eta-invariant η(f) and communication complexity rank r(f) for all generated Boolean functions f exceeds 0.7, with a p-value ≤ 0.05. The criterion is falsified if this correlation coefficient is less than 0.5 or the p-value exceeds 0.1.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `Minimal Eta-Invariant AND Communication Complexity rank` - `Algebraic Geometry local indeterminacy AND Matrix Rank communication complexity` - `eta-invariant 0(r(f)) AND Boolean function f Communication Complexity`

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.7s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def communication_complexity_rank(f):
        n = len(f)
        rank = 0
        for i in range(2**(n-1)):
            a = f[i:i+n//2]
            b = f[i+n//2:i+n]
            if all(a[j] != b[j] for j in range(n//2)):
                rank += 1
        return rank

    def eta_invariant(f):
        n = len(f)
        matrix = [[f[i] ^ f[j] for j in range(n)] for i in range(n)]
        det = determinant(matrix)
        if det == 0:
            return None
        return Fraction(det, 2**n)

    def determinant(matrix):
        n = len(matrix)
        if n == 1:
            return matrix[0][0]
        det = 0
```

```
        for j in range(n):
            submatrix = [row[:j] + row[j+1:] for row in matrix[1:]]
            det += (-1)**j * matrix[0][j] * determinant(submatrix)
        return det

    n_values = [5, 10, 15, 20, 30, 40]
    eta_values = []
    rank_values = []

    for n in n_values:
        f = generate_boolean_function(n)
        rank = communication_complexity_rank(f)
        eta = eta_invariant(f)
        if eta is not None:
            eta_values.append(eta)
            rank_values.append(rank)

    if len(eta_values) < 30 or len(rank_values) < 30:
        return {
            "metric_name": "communication_complexity_rank",
            "metric_value": None,
            "instances_tested": len(eta_values),
            "n_max": max(n_values),
            "conjecture_holds": False,
            "counterexample": "not_enough_instances"
        }

    correlation_coefficient = pearson_correlation(eta_values, rank_values)
    p_value = t_statistic(correlation_coefficient, len(eta_values) - 2)

    return {
        "metric_name": "communication_complexity_rank",
        "metric_value": correlation_coefficient,
        "instances_tested": len(eta_values),
        "n_max": max(n_values),
        "conjecture_holds": correlation_coefficient >= 0.7 and p_value <= 0.05,
        "counterexample": "" if correlation_coefficient >= 0.7 else f"correlation_coefficient={cor
    }

def pearson_correlation(x, y):
    n = len(x)
    mean_x = sum(x) / n
    mean_y = sum(y) / n
    cov_xy = sum((x[i] - mean_x) * (y[i] - mean_y) for i in range(n)) / n
    std_dev_x = math.sqrt(sum((x[i] - mean_x)**2 for i in range(n)) / n)
    std_dev_y = math.sqrt(sum((y[i] - mean_y)**2 for i in range(n)) / n)
    return cov_xy / (std_dev_x * std_dev_y)

def t_statistic(r, n):
    if r == 0:
        return 0
    return r * math.sqrt((n - 2) / (1 - r**2))

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [37, 61, 73, 89, 97, 101, 103, 107, 109, 113, 127, 1
    results = []
```

```
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all(result["conjecture_holds"] for result in results):
        mean_value = sum(result["metric_value"] for result in results) / len(results)
        std_dev = math.sqrt(sum((result["metric_value"] - mean_value)**2 for result in results) /
        support_fraction = 1.0
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_dev} support_fraction={support_fracti
    elif any(not result["conjecture_holds"] and result["counterexample"] != "" for result in resul
        counterexample = next(result["counterexample"] for result in results if not result["conjec
        first_failing_seed = next(result["seed"] for result in results if not result["conjecture_h
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
    else:
        support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(result
        print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b380921c.py", line 105, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b380921c.py", line 58, in run_trial
    rank = communication_complexity_rank(f)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b380921c.py", line 30, in communication
    if all(a[j] != b[j] for j in range(n//2)):
       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b380921c.py", line 30, in <genexpr>
    if all(a[j] != b[j] for j in range(n//2)):
                   ~^^^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means that the Pearson correlation coefficient and p-value could not be calculated to verify the conjecture. | next: Re-run the test without errors to calculate the Pearson correlation coefficient and p-value for verification.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15719
2	preregistration	ollama_remote	glm4:latest	0	0	16603
3	novelty	ollama_remote	glm4:latest	0	0	22216
4	novelty	ollama_remote	glm4:latest	0	0	26506
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18228
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	36716
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	35999
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19143
9	judge	ollama_remote	glm4:latest	0	0	47759

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 238890 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/b6ce94d7be22.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b6ce94d7be22.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/b6ce94d7be22.tar.gz` (if generated)